

과제 설명 영상

<https://www.youtube.com/watch?v=FutONBF37J0>

HTTP Test (with Python Socket)

컴퓨터네트워크 과제

TCP 기반 Server / Client HTTP 통신 구현

□ 목적

- TCP 기반 소켓 통신을 활용하여 HTTP Request / Response 동작 원리를 이해한다.
- Client 가 HTTP 형식의 요청 메시지를 직접 생성하고, Server 가 요청에 따라 응답 메시지를 구성하도록 구현한다.
- Wireshark 를 사용하여 구현한 HTTP 패킷 형식을 확인한다.

□ 목표

1. TCP 기반 Server, Client 프로그램 구현
2. Client 에서 GET / HEAD / POST / PUT Request 요청
3. Server 에서 Client Request 에 따라 HTTP Response 구성
4. Method-Response 조합 Case 5 개 이상 수행
5. Wireshark 로 HTTP format 캡처
6. Server 는 UTM Ubuntu VM, Client 는 macOS 에서 분리 실행

□ 실행 방법

GitHub 에서 소스 코드 다운로드

```
$ git clone https://github.com/hohyun-nam/computernetwork_http.git
$ cd computernetwork_http
```

Server 실행

```
$ cd server
$ python3 server.py
```

Client 실행

```
$ cd client
$ python3 client.py --host 192.168.64.2 --port 8080
```

Server 의 IP 주소가 다르면 --host 값을 실제 IP 주소로 변경한다.

□ 유의사항

- Server 를 먼저 실행한 뒤 Client 를 실행한다.
- Client 메뉴에서 a 를 입력하면 전체 Test Case 를 순서대로 실행한다.
- DELETE 200 Case 는 PUT /sample.txt 수행 후 파일이 생성된 상태에서 정상 동작한다.
- Wireshark 필터는 tcp.port == 8080 또는 http 를 사용한다.

□ 개발환경

구분	환경
Server	UTM Ubuntu VM / Python 3 / server.py
Client	macOS / Python 3 / client.py
통신 방식	TCP Socket
Port	8080

□ Test Case 설계

Case	Method	Path	Request 내용	Response
1	GET	/	기본 페이지 요청	200 OK
2	GET	/abc	존재하지 않는 경로 요청	404 Not Found
3	POST	/message	Body: hello professor	201 Created
4	POST	/message	빈 Body 전송	400 Bad Request
5	PUT	/sample.txt	작은 파일 데이터 업로드	200 OK
6	PUT	/large.txt	제한 크기 초과 데이터 업로드	413 Payload Too Large

7	DELETE	/sample.txt	업로드된 파일 삭제	200 OK
8	DELETE	/ghost.txt	없는 파일 삭제 요청	404 Not Found
9	DELETE	/protected.txt	보호 파일 삭제 요청	403 Forbidden
10	HEAD	/	기본 페이지 Header 요청	200 OK

□ 구현 상황

과제 요구사항	적용 여부	구현 파일
TCP 기반 Server 프로그램 작성	O	server/server.py
TCP 기반 Client 프로그램 작성	O	client/client.py
GET Request / Response 구현	O	client.py / server.py
HEAD Request / Response 구현	O	client.py / server.py
POST Request / Response 구현	O	client.py / server.py
PUT Request / Response 구현	O	client.py / server.py
5 개 이상 Case 수행	O	총 10 개 Case
Wireshark HTTP format 캡처 가능	O	TCP 8080 사용

□ TCP 프로그래밍

Server 는 TCP 소켓을 생성하고 0.0.0.0:8080 에 바인딩한 뒤 Client 연결을 기다린다.

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.bind((args.host, args.port))
    server.listen(5)
    conn, addr = server.accept()
```

Client 는 각 요청마다 TCP 소켓을 생성하고 Server IP 와 Port 로 연결한 후 HTTP Request 를 전송한다.

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
    client.connect((host, port))
    client.sendall(request_bytes)
    response = receive_response(client)
```

□ HTTP Protocol

Client 는 Request Line, Header, Body 를 HTTP/1.1 형식에 맞게 직접 구성한다. GET 과 HEAD 요청은 Server 의 www 폴더 안 실제 파일 경로와 연결된다.

```
GET / HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close
```

POST 와 PUT 처럼 Body 가 있는 요청은 Content-Length 와 Content-Type Header 를 포함한다.

```
POST /message HTTP/1.1
Content-Length: 15
Content-Type: text/plain

hello professor
```

Server 는 Request 를 Method 별로 분기하고, 상태 코드와 Header, Body 를 포함한 Response 를 생성한다.

GET 과 HEAD 는 요청 Path 를 server/www 폴더 안의 파일 경로로 바꾸고, 파일이 존재하면 200 OK, 없으면 404 Not Found 를 반환한다.

```
if method == "GET":
    response = handle_get(path)
elif method == "HEAD":
    response = handle_head(path)
elif method == "POST":
    response = handle_post(path, body)
elif method == "PUT":
    response = handle_put(path, body)
```

□ CASE 설명

CASE1: GET / 요청은 기본 경로이므로 server/www/index.html 을 반환하며 200 OK 가 발생한다.

CASE2: GET /abc 요청은 server/www/abc 파일 또는 server/www/abc/index.html 이 없기 때문에 404 Not Found 가 발생한다.

CASE3: POST /message 에 Body 를 전송하면 201 Created 가 발생한다.

CASE4: POST /message 에 빈 Body 를 전송하면 400 Bad Request 가 발생한다.

CASE5: PUT /sample.txt 에 작은 데이터를 전송하면 파일 저장 후 200 OK 가 발생한다.

CASE6: PUT /large.txt 에 큰 데이터를 전송하면 413 Payload Too Large 가 발생한다.

CASE7: DELETE /sample.txt 는 업로드된 파일을 삭제하고 200 OK 가 발생한다.

CASE8: DELETE /ghost.txt 는 없는 파일이므로 404 Not Found 가 발생한다.

CASE9: DELETE /protected.txt 는 보호 파일이므로 403 Forbidden 이 발생한다.

CASE10: HEAD / 요청은 server/www/index.html 이 존재하므로 Header 만 반환하며 200 OK 가 발생한다.

□ WireShark 캡처 내용

캡처 필터: tcp.port == 8080 && http

Capturing from bridge100

tcp.port == 8080 && http

No.	Time	Source	Destination	Protocol	Length	Info
44	5.640847	192.168.64.1	192.168.64.2	HTTP	166	GET / HTTP/1.1
46	5.649469	192.168.64.2	192.168.64.1	HTTP	207	HTTP/1.1 200 OK (text/html)
55	6.656155	192.168.64.1	192.168.64.2	HTTP	169	GET /abc HTTP/1.1
57	6.657532	192.168.64.2	192.168.64.1	HTTP	171	HTTP/1.1 404 Not Found (text/plain)
71	7.678138	192.168.64.1	192.168.64.2	HTTP	235	POST /message HTTP/1.1 (text/plain)
73	7.672582	192.168.64.2	192.168.64.1	HTTP	187	HTTP/1.1 201 Created (text/plain)
82	8.680788	192.168.64.1	192.168.64.2	HTTP	219	POST /message HTTP/1.1
84	8.682344	192.168.64.2	192.168.64.1	HTTP	169	HTTP/1.1 400 Bad Request (text/plain)
93	9.688407	192.168.64.1	192.168.64.2	HTTP	246	PUT /sample.txt HTTP/1.1
95	9.690619	192.168.64.2	192.168.64.1	HTTP	164	HTTP/1.1 200 OK (text/plain)
104	10.697766	192.168.64.1	192.168.64.2	HTTP	336	PUT /large.txt HTTP/1.1
106	10.698616	192.168.64.2	192.168.64.1	HTTP	179	HTTP/1.1 413 Payload Too Large (text/plain)
115	11.786231	192.168.64.1	192.168.64.2	HTTP	179	DELETE /sample.txt HTTP/1.1
117	11.787539	192.168.64.2	192.168.64.1	HTTP	164	HTTP/1.1 200 OK (text/plain)
126	12.715423	192.168.64.1	192.168.64.2	HTTP	178	DELETE /ghost.txt HTTP/1.1
128	12.717756	192.168.64.2	192.168.64.1	HTTP	171	HTTP/1.1 404 Not Found (text/plain)
137	13.723838	192.168.64.1	192.168.64.2	HTTP	182	DELETE /protected.txt HTTP/1.1
139	13.724631	192.168.64.2	192.168.64.1	HTTP	171	HTTP/1.1 403 Forbidden (text/plain)
148	14.732820	192.168.64.1	192.168.64.2	HTTP	167	HEAD / HTTP/1.1
150	14.732775	192.168.64.2	192.168.64.1	HTTP	149	HTTP/1.1 200 OK

> Frame 44: Packet, 166 bytes on wire (1328 bits), 166 bytes captured (1328 bits) on interface 0
> Ethernet II, Src: 86:94:37:ae:96:64 (86:94:37:ae:96:64), Dst: ee:bf:67:e2:5f:2b
> Internet Protocol Version 4, Src: 192.168.64.1, Dst: 192.168.64.2
> Transmission Control Protocol, Src Port: 65453, Dst Port: 8080, Seq: 1, Ack: 1
> Hypertext Transfer Protocol

0000 ee bf 67 e2 5f 2b 86 94 37 ae 96 64 08 00 45 02 --g...+...7...d..E
0010 00 98 00 00 00 00 40 06 00 00 c0 a8 40 01 c0 a8@.....@...
0020 40 02 ff ad 1f 90 8c 99 75 63 8a 08 a7 a1 80 18 @.....uc.....
0030 08 0b 01 df 00 00 01 01 08 0a 63 d9 97 54 b7 5dc..T..J
0040 db 37 47 45 54 20 2f 20 48 54 54 50 2f 31 2e 31 -7GET / HTTP/1.1
0050 00 0a 48 6f 73 74 3a 20 31 39 32 2e 31 36 38 2e --Host: 192.168.
0060 36 34 2e 32 3a 38 38 38 30 0d 0a 55 73 65 72 2d 64.2:8080 -User=
0070 41 67 65 6e 74 3a 20 43 4e 2d 41 73 73 69 67 6e Agent: C N-Assign-
0080 6d 65 6e 74 2d 43 6c 69 65 6e 74 2f 31 2e 30 8d ment-Client/1.0
0090 0a 43 6f 6e 6e 65 63 74 69 6f 6e 3a 20 63 6c 6f -Connect ion: clo

Hypertext Transfer Protocol: Protocol

Packets: 865 · Displayed: 20 (2.3%)

Profile: Default

□ 실행 화면

Server 실행 화면

```
hohyun@hohyun-pc:~/server$ python3 server.py
HTTP Server Started (0.0.0.0:8080)
█
```

Client 실행 화면

```
[namhohyun@namhohyeon-ui-MacBookAir client % python3 client.py
```

```
===== MENU =====
```

- 1. GET 200
 - 2. GET 404
 - 3. POST 201
 - 4. POST 400
 - 5. PUT 200
 - 6. PUT 413
 - 7. DELETE 200
 - 8. DELETE 404
 - 9. DELETE 403
 - 10. HEAD 200
 - a. RUN ALL
 - 0. EXIT
- Select : █

□ 실행 화면

CASE1

Server

```
===== REQUEST HEADER =====  
GET / HTTP/1.1  
Host: 192.168.64.2:8080  
User-Agent: CN-Assignment-Client/1.0  
Connection: close  
  
===== RESPONSE =====  
HTTP/1.1 200 OK  
Content-Length: 58  
Content-Type: text/html  
Connection: close  
  
<html>  
<body>  
<h1>HTTP Server Running</h1>  
</body>  
</html>
```

Client

```
===== 1. GET / -> 200 OK =====  
[Client -> Server]  
GET / HTTP/1.1  
Host: 192.168.64.2:8080  
User-Agent: CN-Assignment-Client/1.0  
Connection: close  
  
[Server -> Client]  
HTTP/1.1 200 OK  
Content-Length: 58  
Content-Type: text/html  
Connection: close  
  
<html>  
<body>  
<h1>HTTP Server Running</h1>  
</body>  
</html>
```

CASE2

Server

```
Client Connected : ('192.168.64.1', 50195)

===== REQUEST HEADER =====
GET /abc HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close

===== RESPONSE =====
HTTP/1.1 404 Not Found
Content-Length: 14
Content-Type: text/plain
Connection: close

Page Not Found
```

Client

```
===== 2. GET /abc -> 404 Not Found =====
[Client -> Server]
GET /abc HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close

[Server -> Client]
HTTP/1.1 404 Not Found
Content-Length: 14
Content-Type: text/plain
Connection: close

Page Not Found
```


CASE3

Server

```
Client Connected : ('192.168.64.1', 50265)

===== REQUEST HEADER =====
POST /message HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close
Content-Length: 15
Content-Type: text/plain

===== REQUEST BODY =====
hello professor

===== RESPONSE =====
HTTP/1.1 201 Created
Content-Length: 32
Content-Type: text/plain
Connection: close

Message Created: HELLO PROFESSOR
```

Client

```
===== 3. POST /message -> 201 Created =====
[Client -> Server]
POST /message HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close
Content-Length: 15
Content-Type: text/plain

hello professor

[Server -> Client]
HTTP/1.1 201 Created
Content-Length: 32
Content-Type: text/plain
Connection: close

Message Created: HELLO PROFESSOR
```

CASE4

Server

```
Client Connected : ('192.168.64.1', 50268)

===== REQUEST HEADER =====
POST /message HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close
Content-Length: 0
Content-Type: text/plain

===== RESPONSE =====
HTTP/1.1 400 Bad Request
Content-Length: 10
Content-Type: text/plain
Connection: close

Empty Body
```

Client

```
===== 4. POST /message empty body -> 400 Bad Request =====
[Client -> Server]
POST /message HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close
Content-Length: 0
Content-Type: text/plain

[Server -> Client]
HTTP/1.1 400 Bad Request
Content-Length: 10
Content-Type: text/plain
Connection: close

Empty Body
```

CASE5

Server

```
Client Connected : ('192.168.64.1', 50271)

===== REQUEST HEADER =====
PUT /sample.txt HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close
Content-Type: application/octet-stream
Content-Length: 10

===== REQUEST BODY =====
small file

===== RESPONSE =====
HTTP/1.1 200 OK
Content-Length: 14
Content-Type: text/plain
Connection: close

Upload Success
```

Client

```
===== 5. PUT /sample.txt -> 200 OK =====
[Client -> Server]
PUT /sample.txt HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close
Content-Type: application/octet-stream
Content-Length: 10

small file

[Server -> Client]
HTTP/1.1 200 OK
Content-Length: 14
Content-Type: text/plain
Connection: close

Upload Success
```

CASE6

Server

```
Client Connected : ('192.168.64.1', 50274)

===== REQUEST HEADER =====
PUT /large.txt HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close
Content-Type: application/octet-stream
Content-Length: 100

===== REQUEST BODY =====
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

===== RESPONSE =====
HTTP/1.1 413 Payload Too Large
Content-Length: 14
Content-Type: text/plain
Connection: close

File Too Large
```

Client

```
===== 6. PUT /large.txt -> 413 Payload Too Large =====
[Client -> Server]
PUT /large.txt HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close
Content-Type: application/octet-stream
Content-Length: 100

AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

[Server -> Client]
HTTP/1.1 413 Payload Too Large
Content-Length: 14
Content-Type: text/plain
Connection: close

File Too Large
```

CASE7

Server

```
Client Connected : ('192.168.64.1', 50277)

===== REQUEST HEADER =====
DELETE /sample.txt HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close

===== RESPONSE =====
HTTP/1.1 200 OK
Content-Length: 14
Content-Type: text/plain
Connection: close

Delete Success
```

Client

```
===== 7. DELETE /sample.txt -> 200 OK =====
[Client -> Server]
DELETE /sample.txt HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close

[Server -> Client]
HTTP/1.1 200 OK
Content-Length: 14
Content-Type: text/plain
Connection: close

Delete Success
```

CASE8

Server

```
Client Connected : ('192.168.64.1', 50280)

===== REQUEST HEADER =====
DELETE /ghost.txt HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close

===== RESPONSE =====
HTTP/1.1 404 Not Found
Content-Length: 14
Content-Type: text/plain
Connection: close

File Not Found
```

Client

```
===== 8. DELETE /ghost.txt -> 404 Not Found =====
[Client -> Server]
DELETE /ghost.txt HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close

[Server -> Client]
HTTP/1.1 404 Not Found
Content-Length: 14
Content-Type: text/plain
Connection: close

File Not Found
```

CASE9

Server

```
Client Connected : ('192.168.64.1', 50283)

===== REQUEST HEADER =====
DELETE /protected.txt HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close

===== RESPONSE =====
HTTP/1.1 403 Forbidden
Content-Length: 14
Content-Type: text/plain
Connection: close

Protected File
```

Client

```
===== 9. DELETE /protected.txt -> 403 Forbidden =====
[Client -> Server]
DELETE /protected.txt HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close

[Server -> Client]
HTTP/1.1 403 Forbidden
Content-Length: 14
Content-Type: text/plain
Connection: close

Protected File
```

CASE10

Server

```
Client Connected : ('192.168.64.1', 50286)

===== REQUEST HEADER =====
HEAD / HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close

===== RESPONSE =====
HTTP/1.1 200 OK
Content-Length: 58
Content-Type: text/html
Connection: close
```

Client

```
===== 10. HEAD / -> 200 OK =====
[Client -> Server]
HEAD / HTTP/1.1
Host: 192.168.64.2:8080
User-Agent: CN-Assignment-Client/1.0
Connection: close

[Server -> Client]
HTTP/1.1 200 OK
Content-Length: 58
Content-Type: text/html
Connection: close
```


□ 전체 소스 코드

client/client.py

```
import argparse
import socket
import time

# 서버 IP와 포트 번호
# server.py는 UTM 우분투에서 실행하고, client.py는 맥에서 실행하므로
# 우분투에서 ip addr로 확인한 VM IP 주소를 기본 접속 주소로 사용한다.
SERVER_IP = "192.168.64.2"
SERVER_PORT = 8080
BUFFER_SIZE = 4096

def receive_response(client):
    # recv()는 한 번 호출했다고 응답 전체가 무조건 다 오는 것이 아니기 때문에
    # 서버가 Connection: close로 연결을 닫을 때까지 반복해서 응답을 받는다.
    response_chunks = []

    while True:
        chunk = client.recv(BUFFER_SIZE)
        if not chunk:
            break
        response_chunks.append(chunk)

    return b"".join(response_chunks)

def send_request(request_bytes, host, port, title):
    # 하나의 테스트 케이스를 실행하는 함수
    # 먼저 내가 만든 HTTP 요청 메시지를 화면에 출력하고,
    # TCP 소켓으로 서버에 보낸 뒤 서버 응답 메시지도 그대로 출력한다.
    print(f"\n===== {title} =====")
    print("[Client -> Server]")
    print(request_bytes.decode(errors="replace").rstrip())

    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
        client.connect((host, port))
        # send()는 데이터 일부만 보낼 수도 있어서 요청 전체 전송을 위해 sendall() 사용
        client.sendall(request_bytes)
        response = receive_response(client)

    print("\n[Server -> Client]")
    print(response.decode(errors="replace").rstrip())
    time.sleep(1)

def make_text_request(method, path, host, port, body=""):
    # GET, HEAD, POST, DELETE처럼 문자열 body를 사용하는 요청을 만든다.
    # HTTP 메시지는 Request Line, Header, 빈 줄, Body 순서로 구성되어야 한다.
    request = (
        f"{method} {path} HTTP/1.1\r\n"
        f"Host: {host}:{port}\r\n"
        "User-Agent: CN-Assignment-Client/1.0\r\n"
        "Connection: close\r\n"
    )

    if method in ("POST", "PUT"):
        body_bytes = body.encode()
        # POST/PUT은 body가 있을 수 있으므로 Content-Length를 넣어
        # 서버가 body를 몇 바이트까지 읽어야 하는지 알 수 있게 한다.
        request += f"Content-Length: {len(body_bytes)}\r\n"
        request += "Content-Type: text/plain\r\n"
```

```

else:
    body_bytes = b""

    request += "\r\n"
    return request.encode() + body_bytes

def make_binary_request(method, path, host, port, body):
    # PUT 요청처럼 파일 데이터나 bytes 데이터를 보내는 경우 사용한다.
    # text 가 아니라 bytes 그대로 붙여야 하므로 header 만 문자열로 만든 뒤 body 와 합친다.
    header = (
        f"{method} {path} HTTP/1.1\r\n"
        f"Host: {host}:{port}\r\n"
        "User-Agent: CN-Assignment-Client/1.0\r\n"
        "Connection: close\r\n"
        "Content-Type: application/octet-stream\r\n"
        f"Content-Length: {len(body)}\r\n"
        "\r\n"
    ).encode()

    return header + body

def get_cases(host, port):
    # 과제에서 확인할 HTTP Method / Response 테스트 케이스들
    # 각 메뉴 번호마다 요청 메시지를 만들고 send_request()로 전송한다.
    return {
        "1": (
            "GET 200",
            lambda: send_request(
                make_text_request("GET", "/", host, port),
                host,
                port,
                "1. GET / -> 200 OK",
            ),
        ),
        "2": (
            "GET 404",
            lambda: send_request(
                make_text_request("GET", "/abc", host, port),
                host,
                port,
                "2. GET /abc -> 404 Not Found",
            ),
        ),
        "3": (
            "POST 201",
            lambda: send_request(
                make_text_request("POST", "/message", host, port, "hello professor"),
                host,
                port,
                "3. POST /message -> 201 Created",
            ),
        ),
        "4": (
            "POST 400",
            lambda: send_request(
                make_text_request("POST", "/message", host, port, ""),
                host,
                port,
                "4. POST /message empty body -> 400 Bad Request",
            ),
        ),
        "5": (
            "PUT 200",
            lambda: send_request(
                make_binary_request("PUT", "/sample.txt", host, port, b"small file"),
                host,
                port,
                "5. PUT /sample.txt -> 200 OK",
            ),
        ),
        "6": (
            "PUT 413",
            lambda: send_request(
                make_binary_request("PUT", "/large.txt", host, port, b"A" * 100),

```

```

        host,
        port,
        "6. PUT /large.txt -> 413 Payload Too Large",
    ),
),
"7": (
    "DELETE 200",
    lambda: send_request(
        make_text_request("DELETE", "/sample.txt", host, port),
        host,
        port,
        "7. DELETE /sample.txt -> 200 OK",
    ),
),
"8": (
    "DELETE 404",
    lambda: send_request(
        make_text_request("DELETE", "/ghost.txt", host, port),
        host,
        port,
        "8. DELETE /ghost.txt -> 404 Not Found",
    ),
),
"9": (
    "DELETE 403",
    lambda: send_request(
        make_text_request("DELETE", "/protected.txt", host, port),
        host,
        port,
        "9. DELETE /protected.txt -> 403 Forbidden",
    ),
),
"10": (
    "HEAD 200",
    lambda: send_request(
        make_text_request("HEAD", "/", host, port),
        host,
        port,
        "10. HEAD / -> 200 OK",
    ),
),
),
}

def print_menu(cases):
    # 사용자가 원하는 테스트 케이스를 하나씩 선택하거나
    # 전체 케이스를 한 번에 실행할 수 있도록 메뉴를 출력한다.
    print("\n===== MENU =====")
    for key, (title, _) in cases.items():
        print(f"{key}. {title}")
    print("a. RUN ALL")
    print("0. EXIT")

def main():
    # 실행할 때 --host, --port 옵션으로 서버 주소를 바꿀 수 있게 했다.
    # VM IP가 달라져도 코드 수정 없이 명령어 옵션만 바꾸면 된다.
    parser = argparse.ArgumentParser(description="TCP socket HTTP client")
    parser.add_argument("--host", default=SERVER_IP, help="server IP address")
    parser.add_argument("--port", type=int, default=SERVER_PORT, help="server port")
    args = parser.parse_args()

    cases = get_cases(args.host, args.port)

    while True:
        print_menu(cases)
        menu = input("Select : ").strip().lower()

        if menu == "0":
            break
        if menu == "a":
            # a를 입력하면 영상 촬영 시 편하도록 전체 케이스를 순서대로 실행한다.
            for _, run_case in cases.values():
                run_case()
            continue

```

```

        if menu in cases:
            cases[menu][1]()
        else:
            print("Wrong Menu")

if __name__ == "__main__":
    main()

```

server/server.py

```

import argparse
import os
import socket

# 서버는 모든 네트워크 인터페이스에서 8080 포트로 대기한다.
# 0.0.0.0으로 열어두면 UTM 우분투 VM 밖의 맥에서도 접속할 수 있다.
HOST = "0.0.0.0"
PORT = 8080
BUFFER_SIZE = 4096
MAX_UPLOAD_SIZE = 30 # PUT 요청 body가 이 크기를 넘으면 413 응답을 보낸다.

# 서버 파일 위치를 기준으로 www, uploads 폴더 경로를 잡는다.
# 실행 위치가 달라도 server.py가 있는 폴더 기준으로 파일을 찾기 위한 처리이다.
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
UPLOAD_DIR = os.path.join(BASE_DIR, "uploads")
WWW_DIR = os.path.join(BASE_DIR, "www")
INDEX_FILE = os.path.join(WWW_DIR, "index.html")

def prepare_files():
    # 서버 실행에 필요한 폴더와 protected.txt 파일을 준비한다.
    # protected.txt는 DELETE 403 Forbidden 케이스를 만들기 위한 파일이다.
    os.makedirs(UPLOAD_DIR, exist_ok=True)
    os.makedirs(WWW_DIR, exist_ok=True)

    protected_file = os.path.join(UPLOAD_DIR, "protected.txt")
    if not os.path.exists(protected_file):
        with open(protected_file, "w", encoding="utf-8") as file:
            file.write("This file is protected.")

def recv_until_header_end(conn):
    # HTTP 요청에서 Header와 Body는 빈 줄(\r\n\r\n)을 기준으로 나뉜다.
    # 그래서 먼저 Header가 끝나는 지점까지 데이터를 받는다.
    data = b""

    while b"\r\n\r\n" not in data:
        chunk = conn.recv(BUFFER_SIZE)
        if not chunk:
            break
        data += chunk

    return data

def parse_headers(header_text):
    # 첫 줄(Request Line)에서 method, path, version을 분리한다.
    # 예: GET / HTTP/1.1
    lines = header_text.split("\r\n")
    method, path, version = lines[0].split()

    # 나머지 Header들은 이름으로 쉽게 찾을 수 있도록 딕셔너리로 저장한다.
    # Content-Length 같은 값을 나중에 꺼내기 위해 소문자 key로 저장한다.
    headers = {}
    for line in lines[1:]:
        if ":" in line:

```

```

        name, value = line.split(":", 1)
        headers[name.strip().lower()] = value.strip()

    return method, path, version, headers

def recv_body(conn, body_start, content_length):
    # Header 뒤에 body 일부가 이미 같이 들어왔을 수 있으므로 body_start 부터 시작한다.
    # 이후 Content-Length 에 적힌 크기만큼 부족한 데이터를 계속 수신한다.
    body = body_start

    while len(body) < content_length:
        chunk = conn.recv(BUFFER_SIZE)
        if not chunk:
            break
        body += chunk

    return body[:content_length]

def make_response(status_line, body=b"", content_type="text/plain", include_body=True):
    # HTTP Response 는 Status Line, Header, 빈 줄, Body 순서로 구성한다.
    # HEAD 요청처럼 body 를 보내면 안 되는 경우 include_body=False 를 사용한다.
    headers = [
        f"Content-Length: {len(body)}",
        f"Content-Type: {content_type}",
        "Connection: close",
    ]
    response = status_line + "\r\n" + "\r\n".join(headers) + "\r\n\r\n"
    response_bytes = response.encode("utf-8")

    if include_body:
        response_bytes += body

    return response_bytes

def get_static_file_path(path):
    # GET/HEAD 요청의 URL 경로를 server/www 폴더 안의 실제 파일 경로로 바꾼다.
    # / 요청은 웹 서버의 기본 페이지처럼 www/index.html 로 처리한다.
    if path == "/":
        return INDEX_FILE

    request_path = path.lstrip("/")
    file_path = os.path.join(WWW_DIR, request_path)

    # 폴더 경로로 요청한 경우에는 그 안의 index.html 을 기본 파일로 사용한다.
    # 예: /abc 요청인데 www/abc 폴더가 있으면 www/abc/index.html 을 찾는다.
    if os.path.isdir(file_path):
        file_path = os.path.join(file_path, "index.html")

    return file_path

def handle_get(path):
    # GET 요청은 www 폴더에서 해당 파일을 찾아서 보내준다.
    # 파일이 실제로 있으면 200 OK, 없으면 404 Not Found 로 응답한다.
    file_path = get_static_file_path(path)
    if file_path and os.path.exists(file_path) and os.path.isfile(file_path):
        with open(file_path, "rb") as file:
            body = file.read()
        return make_response("HTTP/1.1 200 OK", body, "text/html")

    # 파일이 없으면 없는 페이지로 처리한다.
    body = b"Page Not Found"
    return make_response("HTTP/1.1 404 Not Found", body)

```

```

def handle_head(path):
    # HEAD 는 GET 과 같은 방식으로 파일 존재 여부를 확인한다.
    # 단, HTTP 규칙상 응답 body 는 보내지 않고 header 만 보낸다.
    file_path = get_static_file_path(path)
    if file_path and os.path.exists(file_path) and os.path.isfile(file_path):
        with open(file_path, "rb") as file:
            body = file.read()
            return make_response("HTTP/1.1 200 OK", body, "text/html", include_body=False)

    body = b"Page Not Found"
    return make_response("HTTP/1.1 404 Not Found", body, include_body=False)

def handle_post(path, body):
    # POST 는 /message 경로만 처리한다.
    # 다른 경로로 POST 가 오면 없는 경로로 보고 404 를 반환한다.
    if path != "/message":
        return make_response("HTTP/1.1 404 Not Found", b"Page Not Found")

    # Body 가 비어 있으면 보낼 데이터가 없는 요청이므로 400 Bad Request 로 처리한다.
    if not body.strip():
        return make_response("HTTP/1.1 400 Bad Request", b"Empty Body")

    # 정상 Body 가 오면 메시지가 생성된 것으로 보고 201 Created 를 반환한다.
    response_body = b"Message Created: " + body.upper()
    return make_response("HTTP/1.1 201 Created", response_body)

def handle_put(path, body):
    # PUT 으로 받은 데이터는 uploads 폴더에 파일로 저장한다.
    # 이 과제에서는 sample.txt 와 large.txt 경로만 테스트 대상으로 사용한다.
    filename = os.path.basename(path)

    if path not in ("/sample.txt", "/large.txt"):
        return make_response("HTTP/1.1 404 Not Found", b"Upload Path Not Found")

    # 업로드 크기가 MAX_UPLOAD_SIZE 보다 크면 413 Payload Too Large 로 처리한다.
    if len(body) > MAX_UPLOAD_SIZE:
        return make_response("HTTP/1.1 413 Payload Too Large", b"File Too Large")

    upload_path = os.path.join(UPLOAD_DIR, filename)
    with open(upload_path, "wb") as file:
        file.write(body)

    return make_response("HTTP/1.1 200 OK", b"Upload Success")

def handle_delete(path):
    # DELETE 요청은 uploads 폴더 안의 파일을 대상으로 처리한다.
    # 파일이 있으면 삭제하고, 없으면 404 를 반환한다.
    filename = os.path.basename(path)

    # protected.txt 는 삭제 금지 파일로 설정해서 403 Forbidden 케이스를 만든다.
    if filename == "protected.txt":
        return make_response("HTTP/1.1 403 Forbidden", b"Protected File")

    filepath = os.path.join(UPLOAD_DIR, filename)

    if not os.path.exists(filepath):
        return make_response("HTTP/1.1 404 Not Found", b"File Not Found")

    os.remove(filepath)
    return make_response("HTTP/1.1 200 OK", b>Delete Success")

```

```

def handle_client(conn, addr):
    # 클라이언트 하나의 연결에서 HTTP 요청 하나를 처리하는 부분이다.
    # 요청을 읽고, method 에 따라 응답을 만든 뒤 연결을 닫는다.
    print(f"\nClient Connected : {addr}")

    try:
        request = recv_until_header_end(conn)
        if not request:
            return

        header_bytes, body_start = request.split(b"\r\n\r\n", 1)
        header_text = header_bytes.decode(errors="replace")
        method, path, version, headers = parse_headers(header_text)
        content_length = int(headers.get("content-length", "0"))
        body = recv_body(conn, body_start, content_length)

        print("\n===== REQUEST HEADER =====")
        print(header_text)
        if body:
            print("\n===== REQUEST BODY =====")
            print(body.decode(errors="replace"))

        # Method 에 따라 각각의 처리 함수로 분기한다.
        # 여기서 GET/HEAD/POST/PUT/DELETE 케이스별 응답 코드가 결정된다.
        if method == "GET":
            response = handle_get(path)
        elif method == "HEAD":
            response = handle_head(path)
        elif method == "POST":
            response = handle_post(path, body)
        elif method == "PUT":
            response = handle_put(path, body)
        elif method == "DELETE":
            response = handle_delete(path)
        else:
            response = make_response("HTTP/1.1 405 Method Not Allowed", b"Method Not Allowed")

        conn.sendall(response)

        print("\n===== RESPONSE =====")
        print(response.decode(errors="replace").rstrip())

    except Exception as error:
        response = make_response("HTTP/1.1 400 Bad Request", f"Bad Request: {error}".encode())
        conn.sendall(response)
        print(f"Error: {error}")
    finally:
        conn.close()

def main():
    # --host, --port 옵션을 사용하면 실행할 주소와 포트를 바꿀 수 있다.
    # 기본값은 과제 실행용으로 0.0.0.0:8080 을 사용한다.
    parser = argparse.ArgumentParser(description="TCP socket HTTP server")
    parser.add_argument("--host", default=HOST, help="server bind address")
    parser.add_argument("--port", type=int, default=PORT, help="server port")
    args = parser.parse_args()

    prepare_files()

    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
        # 서버를 바로 재실행할 때 포트가 잠겨있는 문제를 줄이기 위한 옵션이다.
        server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        server.bind((args.host, args.port))
        server.listen(5)

        print(f"HTTP Server Started ({args.host}:{args.port})")

        try:
            while True:

```

```
        conn, addr = server.accept()
        handle_client(conn, addr)
    except KeyboardInterrupt:
        print("\nHTTP Server Stopped")
```

```
if __name__ == "__main__":
    main()
```